



深度学习

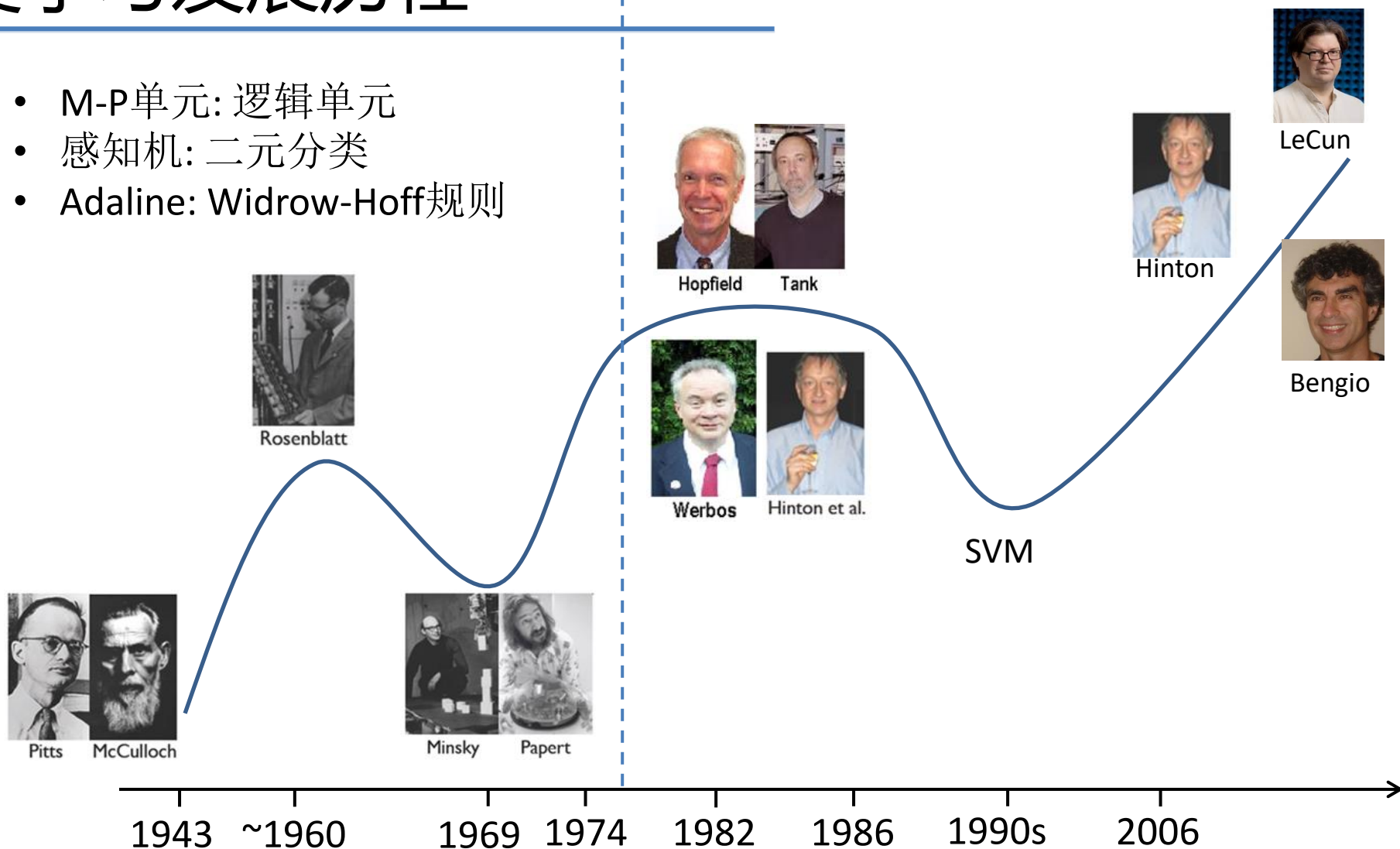
DEEP LEARNING-XIAOLIN HU

Part.04

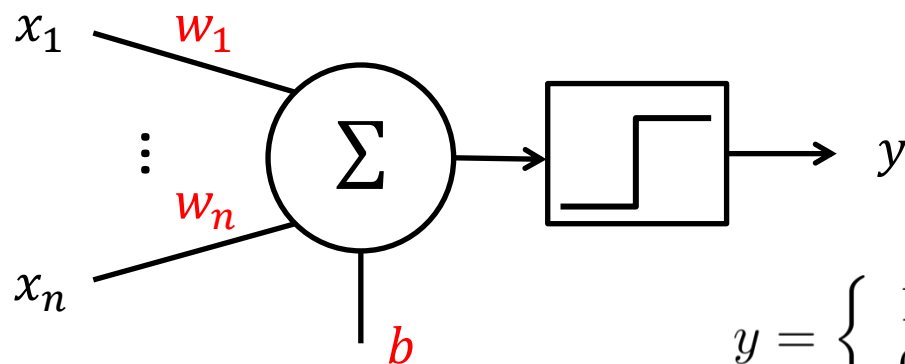
多层感知机

深度学习发展历程

- M-P单元: 逻辑单元
- 感知机: 二元分类
- Adaline: Widrow-Hoff规则



● 回顾: 感知机(Perceptron)



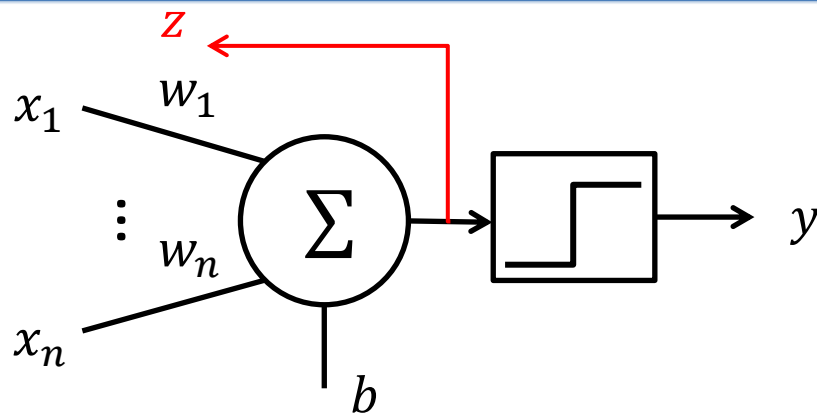
$$y = \begin{cases} 1 & \mathbf{w}^\top \mathbf{x} + b > 0 \\ 0 & \text{else} \end{cases}$$

- 对于每个数据点 $\mathbf{x}^{(j)} \in R^m$ 及对应的标签 $t^{(j)}$
 - 计算输出 $y^{(j)}$
 - 更新参数: $\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} + \eta(t^{(j)} - y^{(j)})\mathbf{x}^{(j)}$;
 $b^{\text{new}} = b^{\text{old}} + \eta(t^{(j)} - y^{(j)})$

其中 η 为学习率且 $\eta > 0$

- 决策边界是一个超平面: $\mathbf{w}^\top \mathbf{x} + b = 0$

● 回顾:自适应线性神经网络(ADALINE)



$$y = \begin{cases} 1 & \mathbf{w}^\top \mathbf{x} + b > 0.5 \\ 0 & \text{else} \end{cases}$$

Or

$$y = \begin{cases} 1 & \mathbf{w}^\top \mathbf{x} + b > 0 \\ -1 & \text{else} \end{cases}$$

- 与感知机的结构一致; 不同训练算法
 - $z = \mathbf{w}^\top \mathbf{x} + b$ 而不是用 y 来更新参数
- 最小化均方误差 $E = \frac{1}{N} \sum_j (t^{(j)} - z^{(j)})^2$. 学习算法:

$$\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} + \eta (t^{(j)} - z^{(j)}) \mathbf{x}^{(j)}$$

$$b^{\text{new}} = b^{\text{old}} + \eta (t^{(j)} - z^{(j)})$$

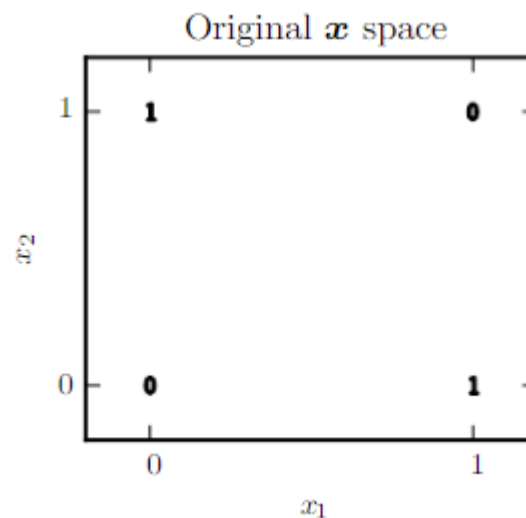
η 为学习率且 $\eta > 0$

- 决策边界是一个超平面: $\mathbf{w}^\top \mathbf{x} + b = 0.5$ 或 $\mathbf{w}^\top \mathbf{x} + b = 0$

● 用ADALINE解决异或(XOR)问题

- 布尔函数:

x_1	x_2	t
0	0	0
0	1	1
1	0	1
1	1	0



- 误差函数 $E = \frac{1}{4} \sum_{j=1}^4 (t^{(j)} - z^{(j)})^2$ 其中 $z^{(j)} = \mathbf{w}^T \mathbf{x}^{(j)} + b$
- 令 $\nabla_{\mathbf{w}} E = 0, \nabla_b E = 0$, 则

$$2w_1 + 2w_2 + 4b = 2$$

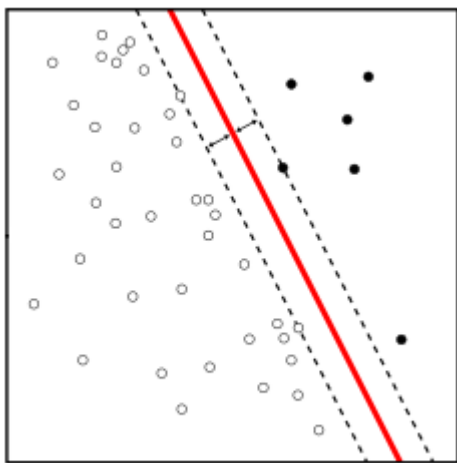
$$2w_1 + w_2 + 2b = 1$$

$$w_1 + 2w_2 + 2b = 1$$

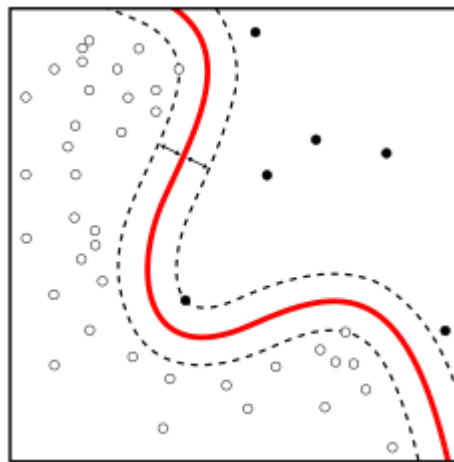
➡ $\mathbf{w}^* = ?, b^* = ?$

● 局限

- 感知机和ADALINE都只能解决线性可分的分类问题



线性可分



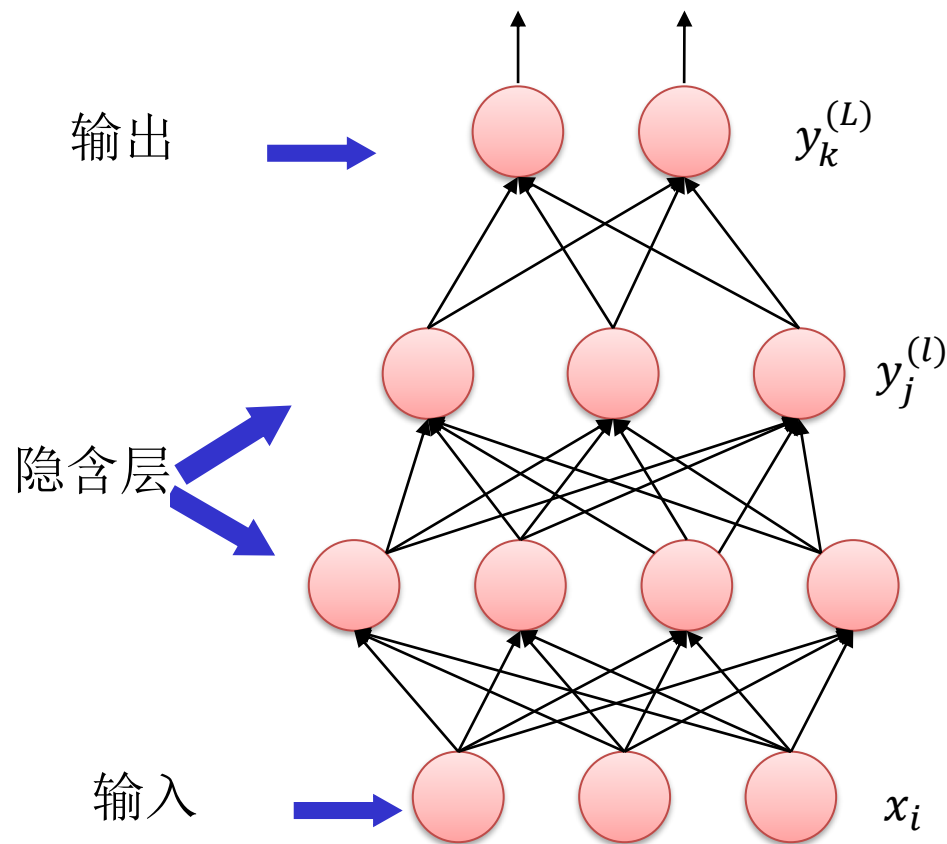
线性不可分

- 这个问题导致了神经网络领域的研究在1960-1970年代退潮 (第一次寒冬)
- 如果所研究的问题是线性不可分的，我们该怎么做？

● 大纲

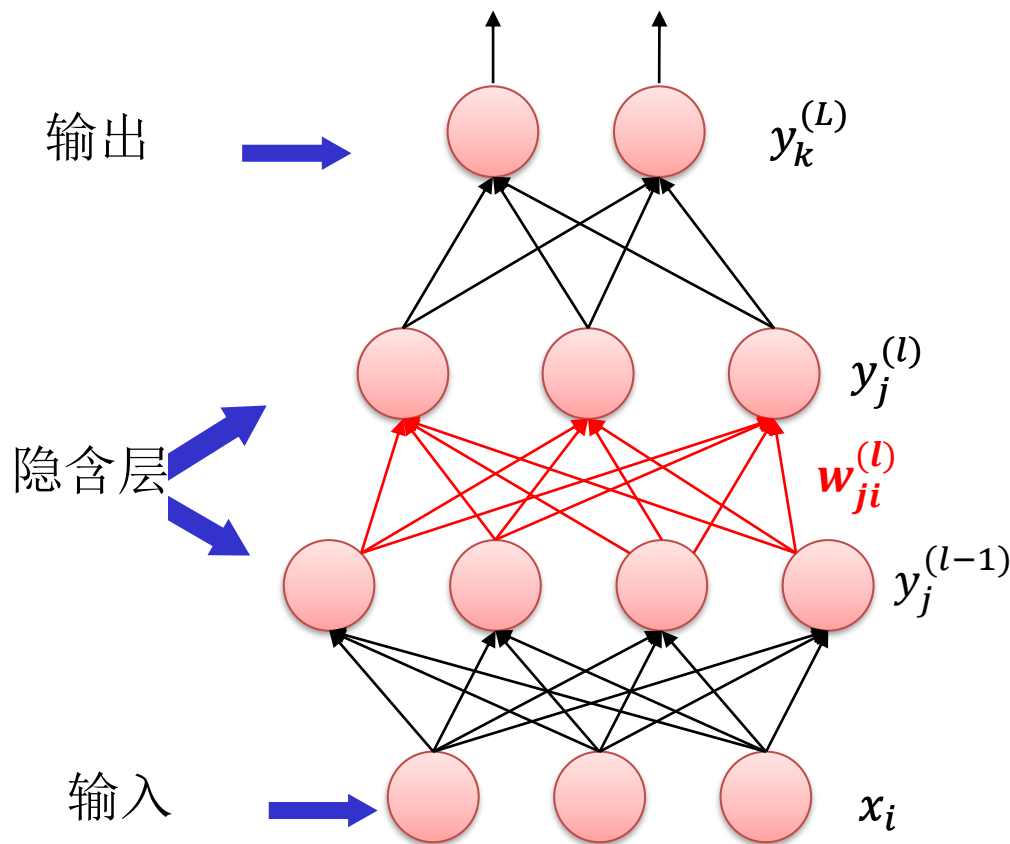
- 多层感知机
 - 前向计算
 - 反向计算
- 层分解
- 训练技巧-1

● 多层感知机(Multi-layer Perceptron, MLP)



- 除了输入共 L 层
- 连接类型:
 - 相邻层之间的神经元两两连接
 - 相邻层之间没有反馈连接
 - 同一层内没有横向连接
- 每个神经元接收前一层神经元的输出，并根据激活函数作出响应

前向传播



- 遍历 $l = 1, \dots, L - 1$ 计算第 l 层的神经元 j 的输入
$$u_j^{(l)} = \sum_i w_{ji}^{(l)} y_i^{(l-1)} + b_j^{(l)}$$
及其输出
$$y_j^{(l)} = f(u_j^{(l)})$$
 $f(\cdot)$ 是激活函数
 - 注意 $y^{(0)} = x$
- $l = L$ 对应分类层

简便起见，这里不区分线性变换和激活函数

● 激活函数

- Logistic函数

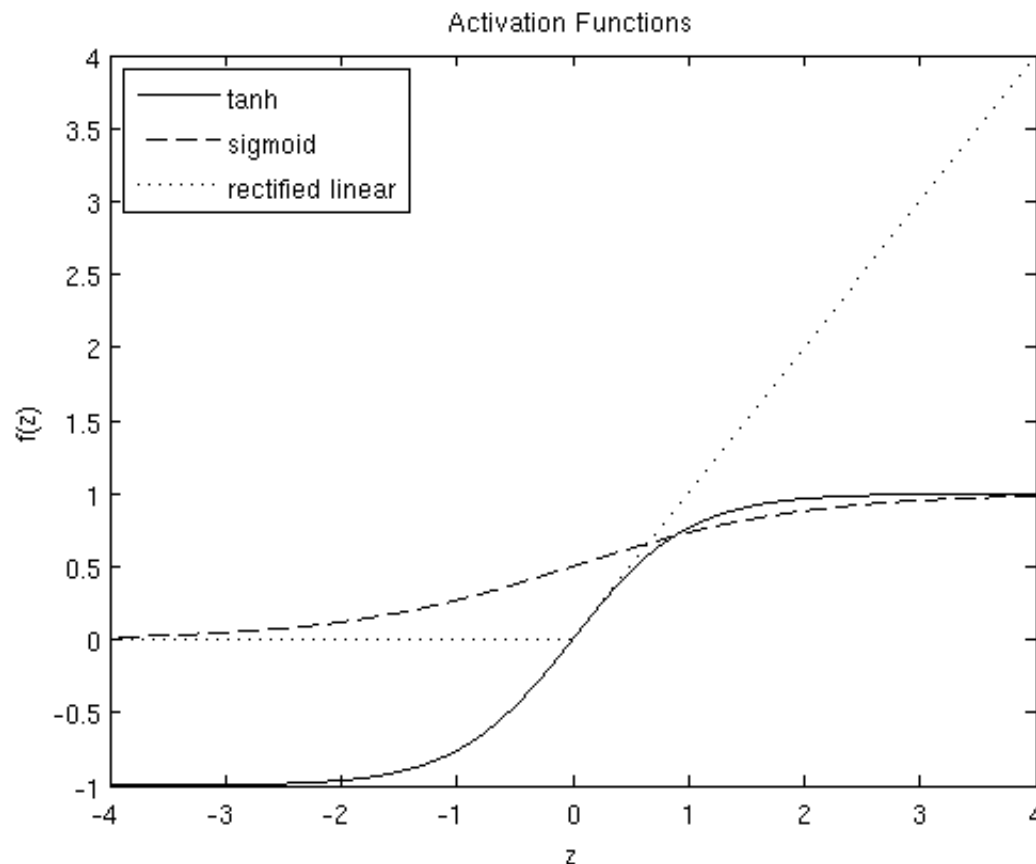
$$f(z) = \frac{1}{1 + \exp(-z)}$$

- 双曲正切函数

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- 整流线性激活函数
(ReLU)

$$f(z) = \max(0, z)$$



● 激活函数

- Logistic函数

$$f(z) = \frac{1}{1 + \exp(-z)}$$

梯度



$$f'(z) = f(z)(1 - f(z))$$

- 双曲正切函数

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

梯度



$$f'(z) = 1 - f(z)^2$$

- 整流线性激活函数
(ReLU)

$$f(z) = \max(0, z)$$

梯度



$$f'(z) = \begin{cases} 1, & \text{if } z \geq 0, \\ 0, & \text{else} \end{cases}$$

● 向量形式的前向计算

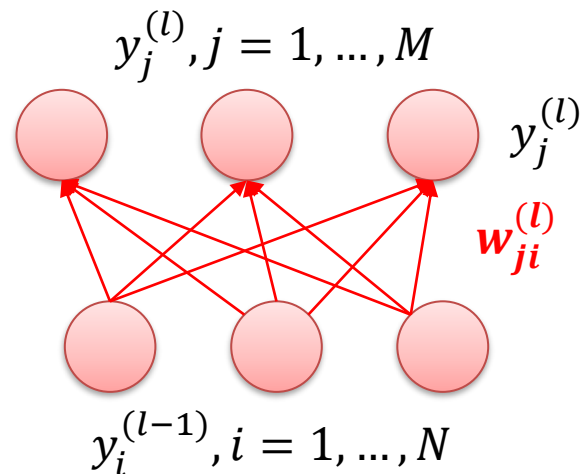
- 如果前一层有 N 个神经元, 当前层有 M 个神经元, 则定义参数矩阵和偏置向量如下:

$$\mathbf{W}^{(l)} = \begin{pmatrix} w_{11} & \cdots & w_{1N} \\ \vdots & \vdots & \vdots \\ w_{M1} & \cdots & w_{MN} \end{pmatrix} \quad \mathbf{b}^{(l)} = \begin{pmatrix} b_1 \\ \vdots \\ b_M \end{pmatrix}$$

- 对于 $l = 1, \dots, L - 1$

$$\mathbf{u}^{(l)} = \mathbf{W}^{(l)} \mathbf{y}^{(l-1)} + \mathbf{b}^{(l)} \text{ and } \mathbf{y}^{(l)} = \mathbf{f}(\mathbf{u}^{(l)})$$

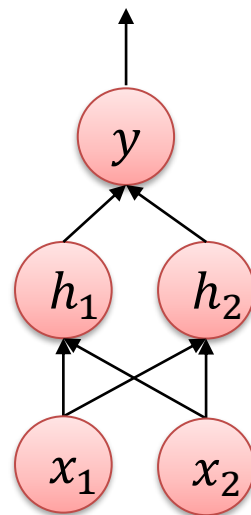
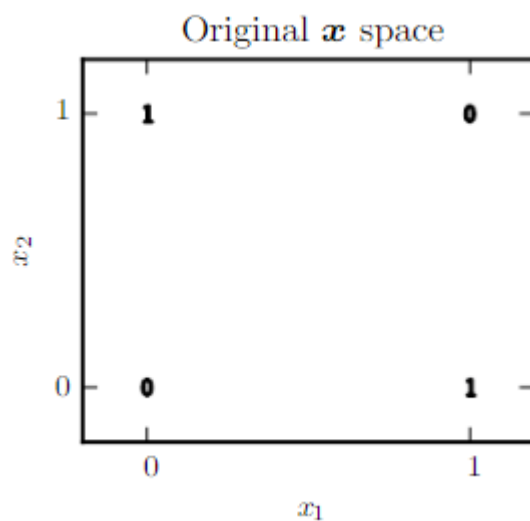
其中 $\mathbf{W}^{(l-1)} \in R^{M \times N}$, $\mathbf{b}^{(l)} \in R^M$, $\mathbf{u}^{(l)}, \mathbf{y}^{(l)} \in R^M$, $\mathbf{y}^{(l-1)} \in R^N$



● 再看异或(XOR)问题

- 布尔函数:

x_1	x_2	t
0	0	0
0	1	1
1	0	1
1	1	0



$$y = \text{ReLU}(\mathbf{w}^\top \mathbf{h} + b)$$

$$\mathbf{w} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, b = 0$$

$$\mathbf{h} = \text{ReLU}(\mathbf{V}\mathbf{x} + \mathbf{c})$$

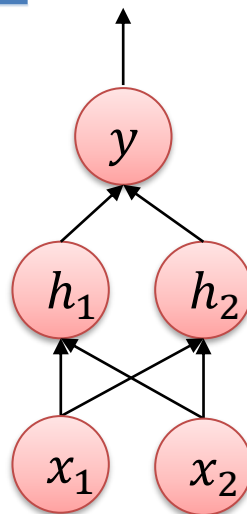
$$\mathbf{V} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \mathbf{c} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$$

什么是 $\mathbf{h}^{(n)}$?

● 再看异或(XOR)问题

- 布尔函数:

x_1	x_2	t
0	0	0
0	1	1
1	0	1
1	1	0

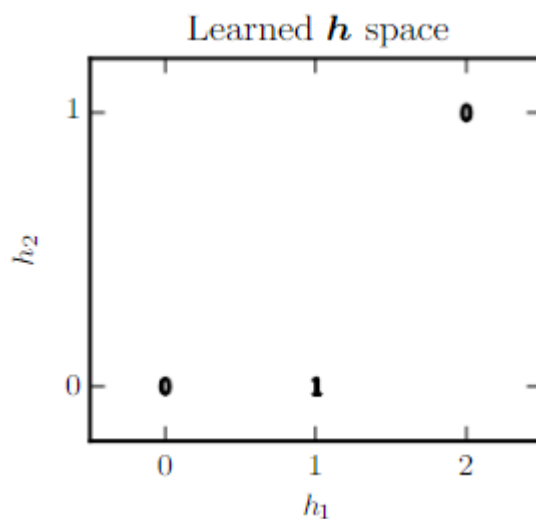
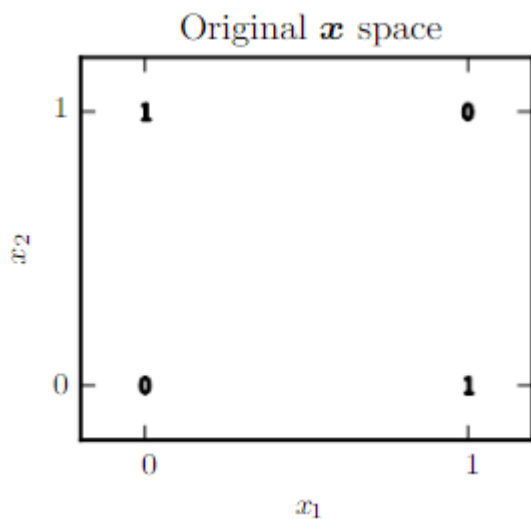


$$y = \text{ReLU}(\mathbf{w}^\top \mathbf{h} + b)$$

$$\mathbf{w} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, b = 0$$

$$\mathbf{h} = \text{ReLU}(\mathbf{V}\mathbf{x} + \mathbf{c})$$

$$\mathbf{V} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \mathbf{c} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$$



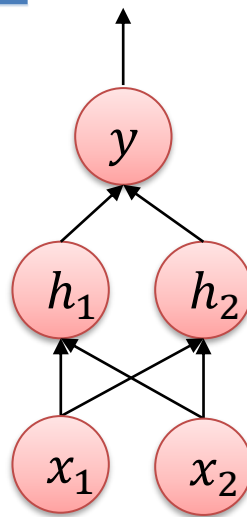
什么是 $\mathbf{h}^{(n)}$?

什么是 $y^{(n)}$?

● 再看异或(XOR)问题

- 布尔函数:

x_1	x_2	t	y
0	0	0	0
0	1	1	1
1	0	1	1
1	1	0	0

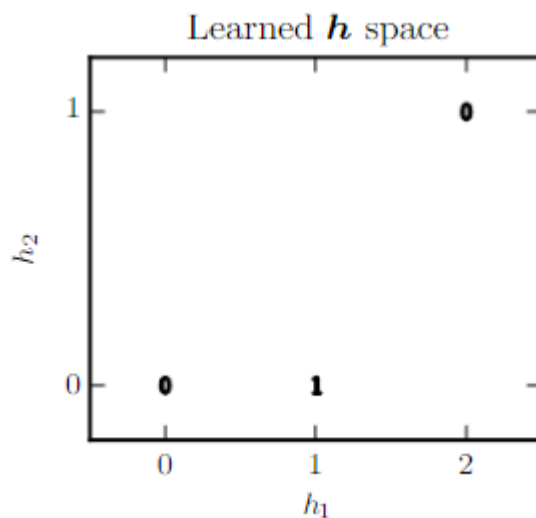
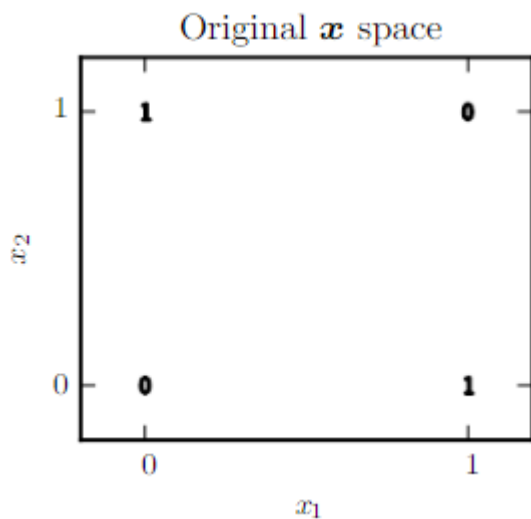


$$y = \text{ReLU}(\mathbf{w}^\top \mathbf{h} + b)$$

$$\mathbf{w} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, b = 0$$

$$\mathbf{h} = \text{ReLU}(\mathbf{V}\mathbf{x} + \mathbf{c})$$

$$\mathbf{V} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \mathbf{c} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$$



什么是 $\mathbf{h}^{(n)}$?

什么是 $y^{(n)}$?

● 大纲

- 多层感知机
 - 前向计算
 - 反向计算
- 层分解
- 训练技巧-1

● 反向传播中的损失函数

- 损失函数

$$E = \frac{1}{N} \sum_{n=1}^N E^{(n)}$$

其中 $E^{(n)}$ 是每个样本 n 的损失函数

- 均方误差, 或欧式距离

$$E^{(n)} = \frac{1}{2} \sum_{k=1}^K (t_k - y_k^{(L)})^2, \quad y_k^{(L)} = \frac{1}{1 + \exp(-\mathbf{w}_k^{(L)\top} \mathbf{y}^{(L-1)} - b_k^{(L)})}$$

是否能用
ReLU?

- 交叉熵误差

$$E^{(n)} = - \sum_{k=1}^K t_k \ln y_k^{(L)}, \quad y_k^{(L)} = \frac{\exp(\mathbf{w}_k^{(L)\top} \mathbf{y}^{(L-1)} + b_k^{(L)})}{\sum_{j=1}^K \exp(\mathbf{w}_j^{(L)\top} \mathbf{y}^{(L-1)} + b_j^{(L)})}$$

其中 $\mathbf{t}$ 是目标向量, 格式如同 $(0, 0, \dots, 1, 0, 0)^T$

简便起见, 除了 $E^{(n)}$, 对 x, t, u, y 等变量省略标记 (n) .

● 参数更新

- 参数更新

$$w_{ji}^{(l)} = w_{ji}^{(l)} - \alpha \frac{\partial E}{\partial w_{ji}^{(l)}} \quad b_j^{(l)} = b_j^{(l)} - \alpha \frac{\partial E}{\partial b_j^{(l)}}$$

学习率
↓

- 参数衰减通常作用于 $w_{ji}^{(l)}$ (对 $b_j^{(l)}$ 不必要), 相当于在损失函数加上额外的一项:

$$J = E + \frac{\lambda}{2} \sum_{i,j,l} (w_{ji}^{(l)})^2$$

- w 的参数更新变为:

$$w_{ji}^{(l)} = w_{ji}^{(l)} - \alpha \frac{\partial J}{\partial w_{ji}^{(l)}} = w_{ji}^{(l)} - \alpha \frac{\partial E}{\partial w_{ji}^{(l)}} - \alpha \lambda w_{ji}^{(l)}$$

● 梯度(Gradient)、局部敏感度(local sensitivity)

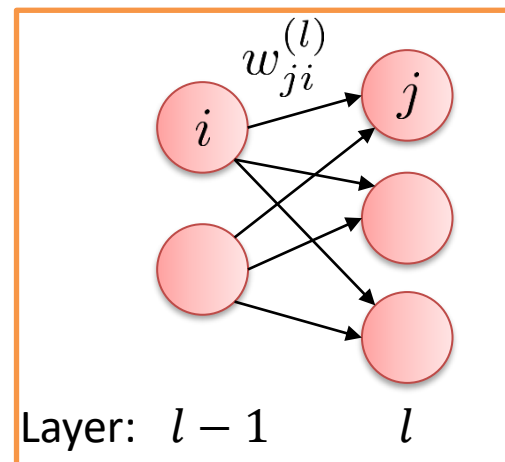
- 局部敏感度
- 对于 $1 \leq l \leq L$

$$\delta_i^{(l)} = \frac{\partial E^{(n)}}{\partial u_i^{(l)}}$$

$$\frac{\partial E^{(n)}}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} \frac{\partial u_j^{(l)}}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} f(u_i^{(l-1)})$$

$$\frac{\partial E^{(n)}}{\partial b_j^{(l)}} = \delta_j^{(l)},$$

$$u_j^{(l)} = \sum_i w_{ji}^{(l)} f(u_i^{(l-1)}) + b_j^{(l)}, \text{ 其中 } f \text{ 是激活函数且}$$
$$f(u_i^{(0)}) = x_i.$$



计算梯度等价于计算各层的局部敏感度！

均方误差(MSE)层的局部敏感度

- 多层感知机最后一层的输出:

$$y_k^{(L)} = f(u_k^{(L)}) = f(\mathbf{w}_k^{(L)\top} \mathbf{y}^{(L-1)} + b_k^{(L)})$$

第L-1层的输出

激活函数 f 可以是

✓ logistic sigmoid

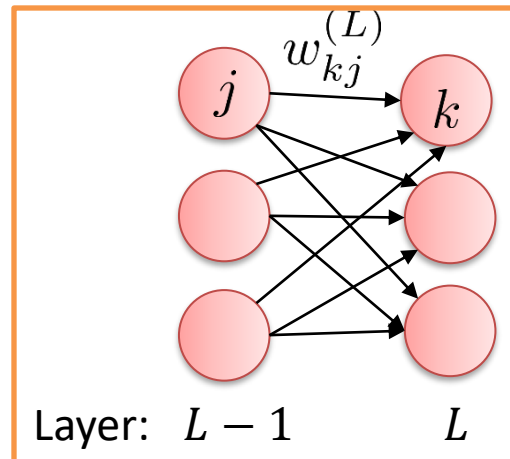
✓ tanh

✓ ReLU

- 每个样本对应的误差
- 局部敏感度

$$E^{(n)} = \frac{1}{2} \sum_{k=1}^K (t_k - y_k^{(L)})^2,$$

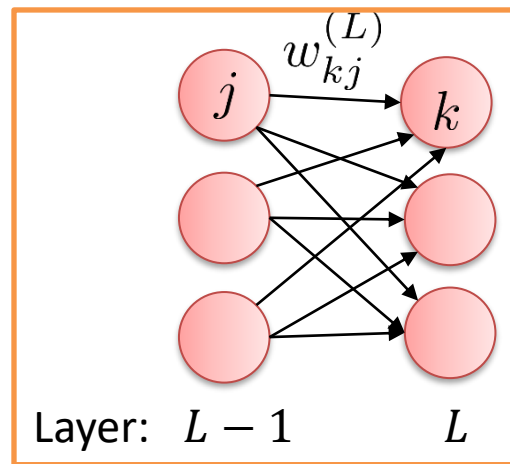
$$\delta_k^{(L)} \triangleq \frac{\partial E^{(n)}}{\partial u_k^{(L)}} = (y_k^{(L)} - t_k) f'(u_k^{(L)})$$



● 回顾: Softmax层的局部敏感度

- 如果多层感知机的最后一层是softmax回归, 则概率函数变为(θ 被替换为 $\mathbf{w}^{(L-1)}$ 和 $b^{(L-1)}$)

第L-1层的输出



$$y_k^{(L)} \triangleq P(t_k = 1 | \mathbf{y}^{(L-1)}) = \frac{\exp(\mathbf{w}_k^{(L)\top} \mathbf{y}^{(L-1)} + b_k^{(L)})}{\sum_{i=1}^K \exp(\mathbf{w}_i^{(L)\top} \mathbf{y}^{(L-1)} + b_i^{(L)})}$$

- 每个样本对应的误差
- 局部敏感度

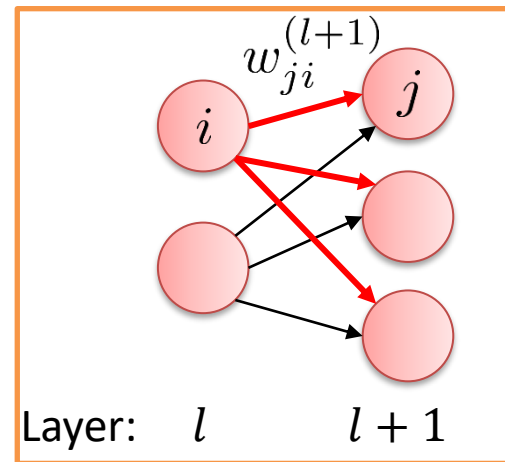
$$E^{(n)}(\theta) = - \sum_{i=1}^K t_i^{(n)} \ln h_i^{(n)}$$

$$\delta_k^{(L)} \triangleq \frac{\partial E^{(n)}}{\partial u_k^{(L)}} = y_k^{(L)} - t_k$$

其它层的局部敏感度

- 局部敏感度
- 对于 $1 \leq l < L$, 例如第 i 个神经元是隐含层神经元, 它将对下一层的所有神经元产生作用, 则局部敏感度为

$$\delta_i^{(l)} = \frac{\partial E^{(n)}}{\partial u_i^{(l)}}$$



$$\delta_i^{(l)} = \frac{\partial E^{(n)}}{\partial u_i^{(l)}} = \sum_j \frac{\partial E^{(n)}}{\partial u_j^{(l+1)}} \frac{\partial u_j^{(l+1)}}{\partial y_i^{(l)}} \frac{\partial y_i^{(l)}}{\partial u_i^{(l)}} = \sum_j \delta_j^{(l+1)} w_{ji}^{(l+1)} f'(u_i^{(l)})$$
$$u_j^{(l+1)} = \sum_i w_{ji}^{(l+1)} y_i^{(l)} + b_j^{(l+1)} \quad y_i^{(l)} = f(u_i^{(l)})$$

f 可以为任意激活函数

因此可以在反向传播中计算 $\delta_i^{(l)}$, $\partial E / \partial W^{(l)}$ 和 $\partial E / \partial b^{(l)}$, 以 $l = L, L-1, \dots, 1$ 的顺序

● 向量形式的反向传播

- 局部敏感度
- 对于输出层 L

$$\delta^{(l)} = \left(\frac{\partial E^{(n)}}{\partial u_1^{(l)}}, \frac{\partial E^{(n)}}{\partial u_2^{(l)}}, \dots \right)^T$$

均方损失: $\delta^{(L)} = (\mathbf{y}^{(L)} - \mathbf{t}) \odot \mathbf{f}'(\mathbf{u}^{(L)})$ 交叉熵损失: $\delta^{(L)} = \mathbf{y}^{(L)} - \mathbf{t}$

其中 $\odot$ 表示逐元素相乘

- 对于隐含层 $1 \leq l < L$

$$\delta^{(l)} = (\mathbf{W}^{(l+1)})^\top \delta^{(l+1)} \odot \mathbf{f}'(\mathbf{u}^{(l)})$$

- 计算梯度 $1 \leq l \leq L$

与 $\mathbf{W}$ 维度一致 $\leftarrow \frac{\partial E^{(n)}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{f}'(\mathbf{u}^{(l-1)}))^\top, \quad \frac{\partial E^{(n)}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$

- 更新参数

—————→ 对 n 平均

$$\mathbf{W}^{(l)} = \mathbf{W}^{(l)} - \frac{\alpha}{N} \sum_n \frac{\partial E^{(n)}}{\partial \mathbf{W}^{(l)}} - \alpha \lambda \mathbf{W}^{(l)}, \quad \mathbf{b}^{(l)} = \mathbf{b}^{(l)} - \frac{\alpha}{N} \sum_n \frac{\partial E^{(n)}}{\partial \mathbf{b}^{(l)}}$$

对每个原本 n

● 梯度消失(Gradient Vanishing)

- 对于隐含层 $1 \leq l < L$

$$\delta^{(l)} = (\mathbf{W}^{(l+1)})^\top \delta^{(l+1)} \odot \mathbf{f}'(\mathbf{u}^{(l)})$$

- 对于logistic函数

$$f(z) = \frac{1}{1 + \exp(-z)}$$

则 $f'(z) = f(z)(1 - f(z)) < 1$

- 对于tanh函数

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

则 $f'(z) = 1 - \tanh^2(z) < 1$

- 对于这两个激活函数, 从第 L 层到第一层, $\delta^{(l)}$ 变得越来越小。
浅层的梯度会逐渐接近于零

$$\frac{\partial E^{(n)}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{f}(\mathbf{u}^{(l-1)}))^\top, \quad \frac{\partial E^{(n)}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$$

ReLU可以缓解这一问题

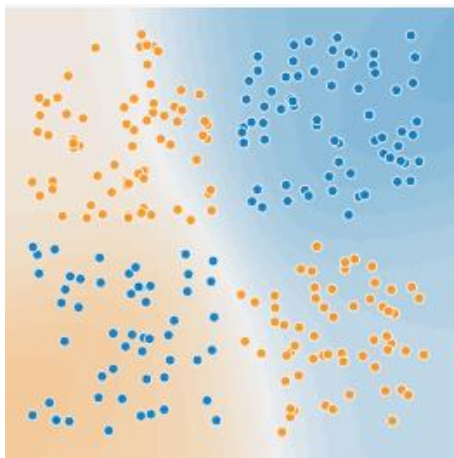
● 实现

- 前向计算
 - 计算 $f(u^l)$ 和 $f'(u^l)$, 对所有的 $l = 1, 2, \dots, L$
- 后向计算
 - 计算 $\delta^{(l)}$ 和 $\partial E / \partial W^{(l)}, \partial E / \partial b^{(l)}$ 对所有的 $l = L, L - 1, \dots, 1$
- 对所有 $l = 1, 2, \dots, L$, 更新 $W^{(l)}$ 和 $b^{(l)}$
- 模块化编程
 - 用类来实现层, 并提供前向和后向计算函数
 - 不同类型的层拥有不同的前向和后向计算函数, 例如输入层、隐含层、softmax输出层、sigmoid输出层等
 - 通过组合不同的模块来构造不同的多层感知机模型

● 实验1: 对二维平面上的点进行分类

- <http://playground.tensorflow.org>

网络配置



- 输入: x_1, x_2
- 隐含层: 1 层, 4 个神经元
- 超参数保持默认值

1. 运行并训练模型
2. 将学习率改为1并重新运行
3. 将学习率改回0.03, 但是使用衰退率分别为0.01, 0.1, 1的L2正则化项
4. 增加第二个隐含层(含2个神经元), 并重新运行

● 实验1: 对二维平面上的点进行分类

- <http://playground.tensorflow.org>



选择一个合适的设定

- 改变隐含层、输入表示、学习率、正则化等

● 实验2: 手写体数字识别

<https://cs.stanford.edu/people/karpathy/convnetjs/demo/mnist.html>

MNIST 数据集

- 60,000 训练图片
- 10,000 图片
- 28x28 灰度图



网络配置

```
layer_defs = [];  
layer_defs.push({type:'input', out_sx:24,  
out_sy:24, out_depth:1});  
layer_defs.push({type:'conv', sx:5, filters:8,  
stride:1, pad:2, activation:'relu'});  
layer_defs.push({type:'pool', sx:2, stride:2});  
layer_defs.push({type:'conv', sx:5, filters:16,  
stride:1, pad:2, activation:'relu'});  
layer_defs.push({type:'pool', sx:3, stride:3});  
layer_defs.push({type:'softmax',  
num_classes:10});
```

红色部分修改为:

```
layer_defs.push({type:'fc', num_neurons:32, activation:'relu'});
```

● 大纲

- 多层感知机
 - 前向计算
 - 反向计算
- 层分解
- 训练技巧-1

● 更灵活的方式

- 输入层或隐含层

$$y_j^{(l)} = f\left(\sum_i w_{ji}^{(l)} y_i^{(l-1)} + b_j^{(l)}\right)$$

可以被解构为两层

- 全连接层: $u_j^{(l)} = \sum_i w_{ji}^{(l)} y_i^{(l-1)} + b_j^{(l)}$
- 激活层: $y_j^{(l)} = f(u_j^{(l)})$

- 平方损失函数层 $E^{(n)} = \frac{1}{2} \left\| \mathbf{f}(\mathbf{u}^{(L)}) - \mathbf{t} \right\|^2$

可以被解构为两层

- 激活层: $y_k^{(L)} = f(u_k^{(L)})$, f 可以是任一函数
- 损失层: $E^{(n)} = \frac{1}{2} \left\| \mathbf{y}^{(L)} - \mathbf{t} \right\|^2$

● 问题

- 考虑平方损失函数

$$E^{(n)} = \frac{1}{2} \left\| \mathbf{f}(\mathbf{W}^{(L)\top} \mathbf{y}^{(L-1)} + \mathbf{b}^{(L)}) - \mathbf{t} \right\|^2$$

需要哪些层来实现?

FC layer + activation layer + Euclidean loss layer
(全连接 + 激活 + 欧式距离损失)

● 更灵活的方式

交叉熵损失层 $E^{(n)} = -\sum_{k=1}^K t_k \ln f(u_k^{(L)})$ 可以被解构为两层

- Softmax层: $y_k^{(L)} = f(u_k^{(L)})$, f 是softmax函数

- 损失层: $E^{(n)} = -\sum_{k=1}^K t_k \ln y_k^{(L)}$

- 并不必要! 为什么?

- 考虑如下误差

$$E^{(n)} = -\sum_{k=1}^K t_k \ln f\left(\sum_i w_{ki}^{(l)} y_i^{(l-1)} + b_k^{(l)}\right)$$

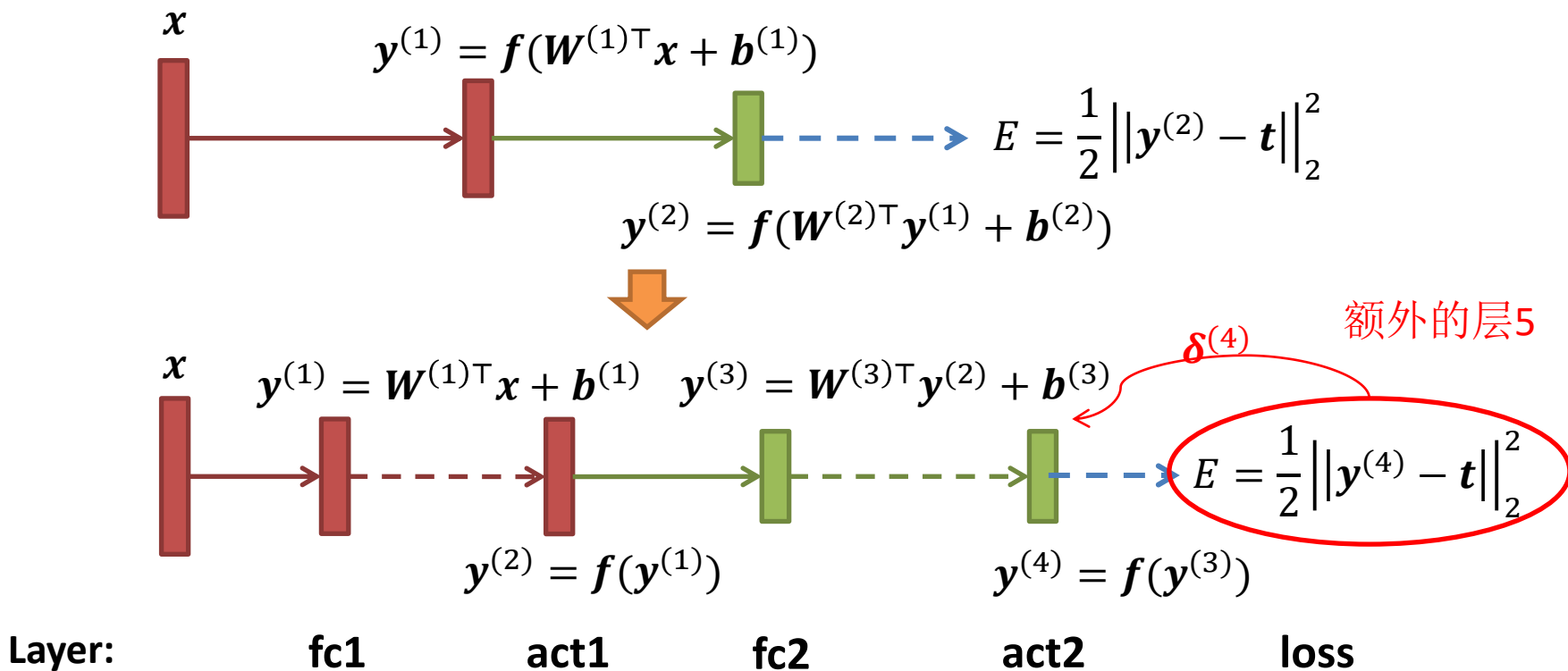
需要哪些层来实现?

FC layer + softmax cross-entropy layer
(全连接 + softmax交叉熵损失)

● 例子1

- 带隐含层的多层感知机，使用平方误差损失

实线:含参数
虚线:不含参数

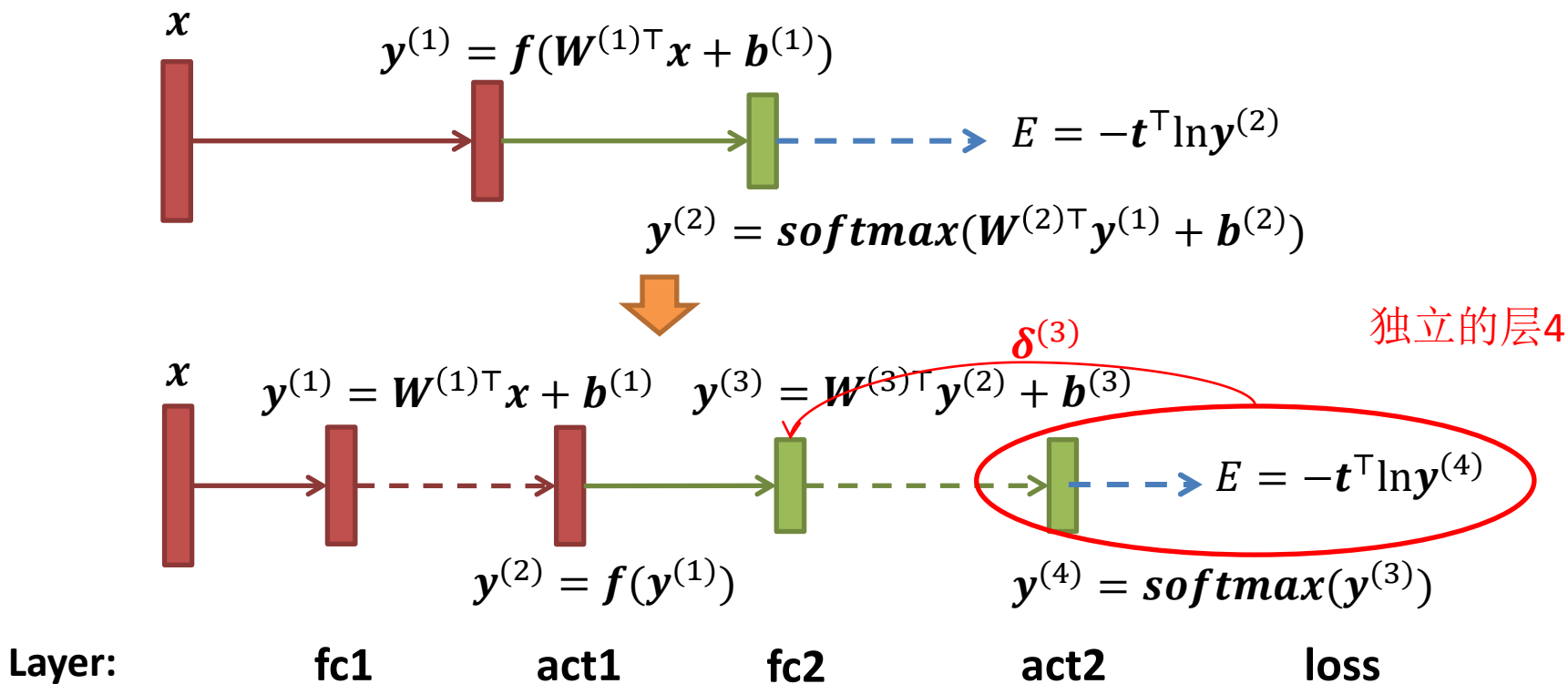


$u^{(l)}$ 和 $y^{(l)}$ 在每一层都保持一致

● 例子2

- 带隐含层的多层感知机，使用平方误差损失

实线:含参数
虚线:不含参数



$u^{(l)}$ 和 $y^{(l)}$ 在每一层都保持一致

● 练习

- 针对以下类型的层推导局部敏感度 δ 以及梯度 $\partial E / \partial w$ 和 $\partial E / \partial b$
 - 欧式距离损失层: $E^{(n)} = \frac{1}{2} \|\mathbf{y}^{(L)} - \mathbf{t}\|^2$
 - 注意 $\delta^{(L)} = \partial E^{(n)} / \partial \mathbf{y}^{(L)}$
 - Softmax交叉熵损失层 $E^{(n)} = -\sum_{k=1}^K t_k \ln f(y_k^{(L)})$
 - 注意 $\delta^{(L-1)} = \partial E^{(n)} / \partial \mathbf{y}^{(L-1)}$
 - 全连接层: $y_j^{(l)} = \sum_i w_{ji}^{(l)} y_i^{(l-1)} + b_j^{(l)}$
 - Sigmoid层: $y_j^{(l)} = f(y_j^{(l-1)})$, f 是sigmoid函数
 - ReLU层: $y_j^{(l)} = f(y_j^{(l-1)})$, f 是ReLU函数

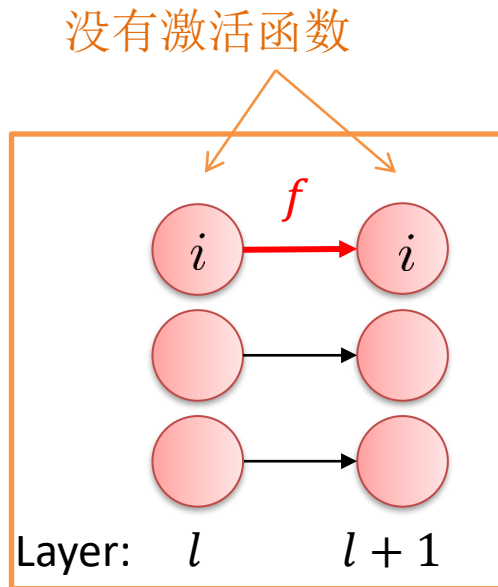
提示

- 假设第 $(l + 1)$ 层是sigmoid激活层:

$$y_i^{(l+1)} = f(y_i^{(l)})$$

其中 f 是sigmoid函数

- 第 l 层的第 i 个神经元只影响第 $l + 1$ 层的第 i 个神经元, 因此



$$\delta_i^{(l)} = \frac{\partial E^{(n)}}{\partial u_i^{(l)}} = \frac{\partial E^{(n)}}{\partial y_i^{(l)}} = \frac{\partial E^{(n)}}{\partial y_i^{(l+1)}} \frac{\partial y_i^{(l+1)}}{\partial y_i^{(l)}} = \delta_i^{(l+1)} f'(y_i^{(l)})$$

不含 w 和 b

- 对其它层可以做类似推导。

● 参考答案

$\mathbf{u}^{(l)}$ 和 $\mathbf{y}^{(l)}$ 在每一层都保持一致

1. 欧式距离损失层

$$\delta^{(L)} = \mathbf{y}^{(L)} - \mathbf{t}$$

2. Softmax交叉熵损失层

$$\delta^{(L-1)} = \mathbf{y}^{(L)} - \mathbf{t}$$

3. 第 l 层为全连接层

↓
Softmax输出

$$\frac{\partial E^{(n)}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{y}^{(l-1)})^\top, \quad \frac{\partial E^{(n)}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}, \quad \delta^{(l-1)} = (\mathbf{W}^{(l)})^\top \delta^{(l)}$$

4. 第 l 层为sigmoid层

$$\delta^{(l-1)} = \delta^{(l)} \odot \mathbf{f}'(\mathbf{y}^{(l-1)}) \quad \text{其中} \quad f'(x) = f(x)(1 - f(x))$$

5. 第 l 层为relu层

$$\delta^{(l-1)} = \delta^{(l)} \odot \mathbf{f}'(\mathbf{y}^{(l-1)}) \quad \text{其中} \quad f'(x) = \begin{cases} 1, & \text{if } x \geq 0 \\ 0, & \text{else.} \end{cases}$$

● 向量形式的反向传播

前向传播: MSE创建 $L + 1$ 层; CE创建 L 层

- 最后一层
 - 欧式距离损失层, $l = L + 1$
 - Softmax交叉熵损失层, $l = L$
- 从最后一层到第一层
 - 第 l 层是全连接层

$$\frac{\partial E^{(n)}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{y}^{(l-1)})^\top, \quad \frac{\partial E^{(n)}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}, \quad \boldsymbol{\delta}^{(l-1)} = (\mathbf{W}^{(l)})^\top \boldsymbol{\delta}^{(l)}$$

- 第 l 层是激活层

$$\boldsymbol{\delta}^{(l-1)} = \boldsymbol{\delta}^{(l)} \odot \mathbf{f}'(\mathbf{y}^{(l-1)}) \quad f(\cdot) \text{ 是激活函数}$$

- 更新参数

$$\mathbf{W}^{(l)} = \mathbf{W}^{(l)} - \frac{\alpha}{N} \sum_n \frac{\partial E^{(n)}}{\partial \mathbf{W}^{(l)}} - \alpha \lambda \mathbf{W}^{(l)}, \quad \mathbf{b}^{(l)} = \mathbf{b}^{(l)} - \frac{\alpha}{N} \sum_n \frac{\partial E^{(n)}}{\partial \mathbf{b}^{(l)}}$$

$$\delta_i^{(l)} \triangleq \frac{\partial E^{(n)}}{\partial y_i^{(l)}}$$

$$\begin{aligned} \boldsymbol{\delta}^{(L)} &= \mathbf{y}^{(L)} - \mathbf{t} \\ \boldsymbol{\delta}^{(L-1)} &= \mathbf{y}^{(L)} - \mathbf{t} \end{aligned}$$

对每个样本 n

——→ 对 n 平均

● 大纲

- 多层感知机
 - 前向计算
 - 反向计算
- 层分解
- 训练技巧-1

● 参数初始化

W 可以从某个分布中采样:

- 高斯分布(Gaussian)
 - 均值为零, 标准差固定(例如0.01)的高斯分布
- Xavier
 - 均值为零, 标准差固定为 $1/\sqrt{n_{\text{in}}}$ 的一种分布, 其中 n_{in} 是当前神经元输入的个数
 - 通常使用高斯分布或均匀分布
- MSRA
 - 均值为零, 标准差固定为 $2/\sqrt{n_{\text{in}}}$ 的高斯分布

● 学习率

- 随机梯度下降中的学习率 α 通常远小于批量梯度下降中的，因为更新过程中存在更大的方差。
- 选择合适的策略
 - 一个标准的做法是选择一个足够小的学习率使得网络开始稳定收敛，在收敛速度变慢后将学习率减半。
 - 另一种更好的策略是每一轮训练后都在一组留出数据集(held out set)上评估模型，如果相邻两轮训练中目标函数的改变量小于某个阈值，则减小学习率。
 - 另一种常用策略是在第 t 次迭代(iteration)中将学习率修改为 $\frac{a}{b+t}$ ，其中 a 是初始的学习率， b 控制学习率何时开始衰减。

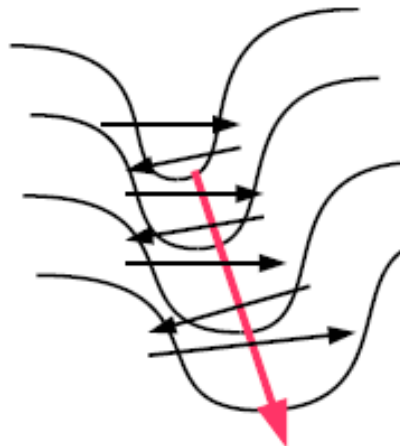
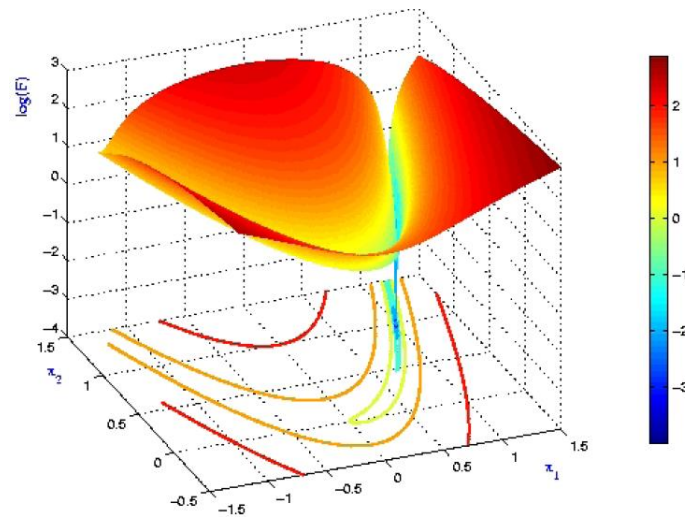
● 训练样本的顺序

- 如果样本的训练顺序固定，梯度的计算可能会有偏差，并导致收敛情况变差
- 通常在每轮训练前都要随机打乱(random shuffle)数据集

● 病态的曲率(Pathological Curvature)

- 目标函数的形式通常是狭长且浅的沟壑
 - 就像 Rosenbrock 函数一样
- 在深度网络中，目标函数的局部最优点经常以这种形式呈现，所以标准的随机梯度下降算法通常会在“沟壑”两侧反复震荡

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2$$



黑色箭头: 梯度下降的路径

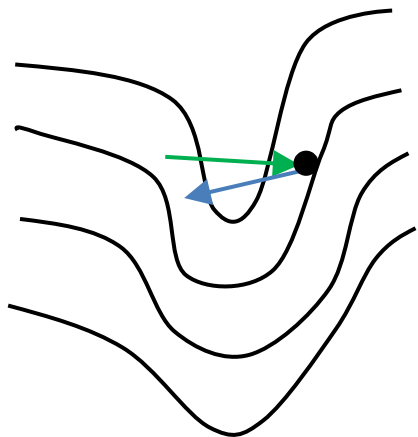
● 动量(Momentum)

- 动量(Momentum)是一种迫使目标函数沿着“沟壑”快速下降的方法
- 动量的更新过程:

$$\begin{aligned} \boldsymbol{v} &= \gamma \boldsymbol{v} - \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}; \boldsymbol{x}^{(i)}, \boldsymbol{t}^{(i)}) \\ \boldsymbol{\theta} &= \boldsymbol{\theta} + \boldsymbol{v} \end{aligned}$$

- $\boldsymbol{v}$ 是当前的速度向量
- $\gamma \in (0,1]$ 决定了之前迭代的梯度以多大程度参与到当前的更新中
- 一种策略: γ 设为 0.5 直到收敛开始稳定, 然后增大到 0.9 或更大

● 例子

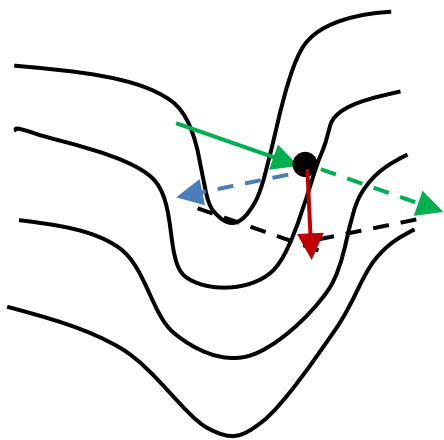


$$\text{令 } g = -\nabla_{\theta} J(\theta; x^{(i)}, t^{(i)})$$

绿色箭头: 上一次迭代中 θ 的改变量

- 标准梯度下降:

$$\Delta\theta = \alpha g$$



- 考虑动量的梯度下降:

$$\Delta\theta = \gamma v + \alpha g$$

$\Delta\theta$ 更符合“沟壑”的下降方向

● 总结

- 多层感知机
 - 前向计算
 - 后向计算: 反向传播
- 层分解
 - 全连接层, sigmoid层, ReLU层, 损失函数层
- 训练技巧-1
 - 参数初始化, 学习率, 训练样本顺序, 动量

● 延伸阅读

- Liao, Leibo, Poggio (2015),
How important is weight symmetry in backpropagation?
[AAAI](#)
- Smith (2018)
Cyclical learning rates for training neural networks
[arXiv:1506.01186v6](#)